TareasYA

SPEC y plan técnico del sistema

Versión del documento: 1.1 (corresponde a la versión de software v0.11.0) - 3 de noviembre de 2026

Autor: Santino Crespo - 6.º 2.ª - E.T. N.º 35 Ing. Eduardo Latzina

Elemento	Decisión
Tipo de sistema	Aplicación web de gestión de tareas escolares (SPA + API REST)
Frontend	React 18 + Vite 5, CSS propio (responsive, modo oscuro)
Backend	Node.js 22 + Express 5, arquitectura en capas
Persistencia	Archivo JSON con escritura atómica (backend) o localStorage (modo local)
Pruebas	Vitest (22 tests, frontend) y node:test (31 tests, backend), con desarrollo guiado por tests
Flujo de trabajo	Ciclo Rojo-Verde-Refactor con agentes de IA (tdd y revisor)

1. Objetivo y problema

Los estudiantes de nivel secundario y terciario organizan sus entregas en cuadernos, mensajes sueltos o el chat del curso, y las fechas se pierden. TareasYA centraliza las tareas y trabajos prácticos por materia, prioridad y fecha límite, y avisa antes de que venzan.

Usuario objetivo

- Estudiante que cursa varias materias a la vez y entrega trabajos con fechas distintas.
- Uso individual, desde la computadora o el celular, sin necesidad de crear una cuenta.

Criterios de éxito

- Cargar una tarea completa en menos de 30 segundos.
- Ver de un vistazo qué vence primero y qué está vencido.
- No perder datos ante un cierre inesperado del servidor o del navegador.
- La aplicación sigue siendo usable si el servidor no está disponible (modo local).

2. Alcance

Incluido en esta versión	Fuera de alcance (por ahora)
Crear, listar, editar y eliminar tareas	Cuentas de usuario y autenticación
Estados: pendiente, en curso, completada	Sincronización entre dispositivos
Prioridad, materia, fecha límite, favoritas	Tareas compartidas o grupales
Búsqueda, filtros y orden	Adjuntos y archivos
Resumen y aviso de vencimientos (48 h)	Recordatorios por correo o mensajería
Notificaciones del navegador	Aplicación móvil nativa
Modo local sin servidor	

3. Requisitos funcionales

ID	Requisito	Prioridad
RF-01	El usuario puede crear una tarea con título, materia, fecha límite y prioridad (descripción opcional).	Alta
RF-02	El sistema rechaza datos inválidos con un mensaje específico por cada error.	Alta
RF-03	No se permiten dos tareas activas con el mismo título y materia (sin distinguir mayúsculas).	Alta
RF-04	El usuario puede editar cualquier campo de una tarea existente.	Alta
RF-05	El usuario puede cambiar el estado de una tarea y marcarla como favorita.	Alta
RF-06	El usuario puede eliminar una tarea, con confirmación previa.	Media
RF-07	El usuario puede buscar por título o materia y filtrar por materia, estado, prioridad y favoritas.	Alta
RF-08	El usuario puede ordenar por fecha límite, prioridad o antigüedad.	Media
RF-09	La interfaz muestra un resumen (pendientes, en curso, completadas, vencidas).	Media
RF-10	La interfaz destaca las tareas vencidas y avisa de las que vencen en 48 horas.	Alta
RF-11	Con permiso del usuario, el navegador notifica las tareas por vencer.	Baja
RF-12	Si el servidor no responde, la aplicación funciona con datos locales y lo informa.	Media
RF-13	La API entrega un resumen: total, por estado, vencidas y por vencer (48 h).	Media
RF-14	La API lista las tareas activas que vencen en las próximas N horas (1 a 720), la más próxima primero.	Media

4. Requisitos no funcionales

Categoría	Requisito	Cómo se cumple
Integridad	No perder ni corromper datos	Escritura atómica (archivo temporal + rename); base dañada produce error y no una lista vacía
Seguridad	No exponer detalles internos	Manejador central de errores con mensajes genéricos; límite de 100 kb por petición
Validación	Toda entrada se valida en el servidor	Las reglas viven en logic/taskLogic.js (puras y testeadas); el cliente repite las mismas reglas
Mantenibilidad	Cambios localizados	Capas con una sola responsabilidad; la capa de datos se puede reemplazar sin tocar reglas
Usabilidad	Uso fluido en celular	Diseño responsive, elementos táctiles amplios, modo oscuro automático
Accesibilidad	Uso con teclado y lector de pantalla	Etiquetas, roles ARIA (alert, aria-pressed, aria-label) y foco visible
Testeabilidad	Probar sin abrir puertos ni UI	app.js separado de server.js; lógica pura sin React ni Express

5. Arquitectura

Aplicación de dos partes que se comunican por HTTP y JSON. En desarrollo, Vite reenvía las llamadas a `/api` al backend (puerto 3001), por lo que no hace falta configurar CORS.

Backend (capas)

Capa	Archivo	Responsabilidad
Rutas	routes/taskRoutes.js	Asocia método y URL con una función del controller
Controller	controllers/taskController.js	Traduce HTTP a llamadas al service y el resultado a código de estado; sin reglas
Service	services/taskService.js	Reglas de negocio: validar, rechazar duplicadas (409), 404, armar la respuesta
Lógica	logic/taskLogic.js	Funciones puras: validar, crear, aplicar cambios, filtrar y ordenar
Repositorio	repositories/taskRepository.js	Único que sabe dónde se guardan los datos (JSON con escritura atómica)
App / Server	app.js, server.js	app.js arma Express y el manejo de errores; server.js solo abre el puerto

Flujo de una petición

```
Cliente --HTTP--> Ruta --> Controller --> Service --> Lógica (validar)
                                                        |
                                                        +--> Repositorio --> data/tareas.json
Cliente <--JSON-- Controller <-- resultado {ok, status, tarea | error}
```

Frontend

Módulo	Responsabilidad
App.jsx	Orquesta: carga datos, estado global de la pantalla, avisos y notificaciones
components/TaskForm.jsx	Alta y edición; solo estado de UI y llamada a onSubmit
components/TaskFilters.jsx	Búsqueda, filtros y orden (componente controlado)
components/TaskList.jsx	Lista y acciones rápidas por tarea
logic/taskLogic.js	Lógica pura: validación, filtros, orden, vencimientos, resumen
logic/taskClient.js	Puerta única a los datos: API si hay servidor, localStorage si no
logic/taskStorage.js	Lectura y escritura en localStorage (modo local)

6. Modelo de datos

Campo	Tipo	Reglas
id	texto	Generado por el sistema; no editable
titulo	texto	Obligatorio, hasta 100 caracteres, se recortan espacios
descripcion	texto	Opcional, hasta 500 caracteres
materia	texto	Obligatoria
fechaLimite	fecha ISO 8601 (UTC)	Obligatoria y válida; se muestra en hora local
prioridad	baja | media | alta	Obligatoria
estado	pendiente | en curso | completada	Inicia en pendiente
favorito	booleano	Inicia en false
fechaCreacion	fecha ISO	Asignada al crear; no editable
fechaActualizacion	fecha ISO	Se actualiza en cada edición

7. Reglas de negocio

- **Duplicadas:** no puede haber dos tareas *activas* con igual título y materia (sin distinguir mayúsculas ni espacios al borde). Las completadas no cuentan, para poder repetir una tarea periódica.
- **Edición:** solo se aceptan campos editables; id y fechas de sistema se rechazan. La regla de duplicadas se aplica también al editar, ignorando la propia tarea.
- **Vencida:** tarea no completada con fecha límite pasada. **Por vencer:** no completada con fecha dentro de las próximas 48 horas.
- **Orden por defecto:** fecha límite, la más próxima primero.

8. API REST

Base: `/api`. Cuerpos en JSON. Los errores devuelven `{ "error": "mensaje" }`.

Método y ruta	Descripción	Respuestas
GET /salud	Comprobación de disponibilidad	200
GET /tareas	Lista. Query: q, materia, estado, prioridad, favorito=true, orden=fecha|prioridad|creacion	200
POST /tareas	Crea una tarea	201, 400 inválida, 409 duplicada
GET /tareas/resumen	Contadores: total, pendientes, enCurso, completadas, vencidas, porVencer	200
GET /tareas/proximas	Tareas activas que vencen pronto. Query: horas (entero 1 a 720, por defecto 48)	200, 400 horas inválidas
GET /tareas/:id	Obtiene una tarea	200, 404
PATCH /tareas/:id	Edita campos (parcial)	200, 400, 404, 409
DELETE /tareas/:id	Elimina una tarea	204, 404

Código	Cuándo
400	Datos inválidos, campo no editable, parámetro horas fuera de rango, JSON mal formado
404	Tarea o ruta inexistente
409	Ya existe una tarea activa con ese título y materia
413	Cuerpo mayor a 100 kb
500	Error interno o base de datos dañada (mensaje genérico)

Ejemplo

```
POST /api/tareas
{ "titulo": "TP de Física", "materia": "Física",
  "fechaLimite": "2026-12-01T20:00", "prioridad": "alta" }

201 Created
{ "id": "mhk3x9...", "estado": "pendiente", "favorito": false, ... }
```

9. Interfaz y modos de funcionamiento

- **Modo API:** al iniciar, `detectarModo()` consulta /api/salud; si responde, todos los datos pasan por el servidor.
- **Modo local:** si no responde, se usa localStorage con las mismas reglas del backend y se muestra un aviso visible.

- **Pantalla:** encabezado, resumen, aviso de por vencer, formulario, filtros y lista. Cada tarea muestra color de prioridad, materia, vencimiento relativo (por ejemplo vence en 3 h) y acciones.
- **Errores:** los mensajes del servidor (por ejemplo 409) se muestran en el formulario o en un aviso general, con role=alert.

10. Seguridad y robustez

Riesgo	Mitigación actual	Pendiente
Datos corruptos en disco	Escritura atómica; si el JSON es inválido se responde 500 sin sobrescribir	Copias de seguridad
Entradas maliciosas o malformadas	Validación de tipo, largo y valores permitidos; campos desconocidos rechazados	Limitar la tasa de peticiones
Fuga de información interna	Mensajes de error genéricos; el detalle solo va al log	Registro estructurado
XSS	React escapa el contenido; no se usa HTML en crudo	Cabeceras de seguridad (CSP)
Acceso no autorizado	No hay autenticación (uso personal local)	Cuentas y sesiones antes de exponer a internet
Escrituras simultáneas	Operaciones sincrónicas dentro de un solo proceso	Base de datos con transacciones

11. Estrategia de pruebas

Nivel	Herramienta	Cantidad	Qué cubre
Unitarias - frontend	Vitest	22	Validación, creación, cambios, duplicadas, filtros, orden, vencimientos, resumen, fechas
Unitarias - backend	node:test	10	Validación completa y parcial, tipos inesperados, filtrar y ordenar, resumen, tareas por vencer
Integración - API	node:test + fetch	21	Códigos 201, 400, 404, 409, 500; filtros; edición; borrado; JSON inválido; base dañada; resumen; próximas con seis valores inválidos

Los tests de la API usan una base de datos temporal (variable TAREASYA_DB) y un puerto aleatorio, por lo que no tocan los datos reales. Estado al cierre de esta versión: 53 de 53 tests correctos (31 backend y 22 frontend) y build de producción exitoso.

Casos límite considerados

- Título con espacios al borde, de 101 caracteres o que no es texto.
- Fecha vacía frente a fecha con formato imposible.
- Duplicada contra una tarea completada, y duplicada creada al editar.
- Id inexistente en GET, PATCH y DELETE.
- Base de datos con JSON inválido.
- Ventana de vencimiento: límite exacto incluido, tarea ya vencida, tarea completada.
- Parámetro horas: texto, cero, negativo, decimal, mayor a 720 y vacío.

12. Flujo de trabajo y agentes de IA

El desarrollo sigue el ciclo **Rojo-Verde-Refactor** y se apoya en dos agentes de IA definidos en `.claude/agents/`.

Fase	Qué se hace	Cuándo termina
Rojo	Se escribe solo el test de un comportamiento nuevo	El test falla por la razón correcta (función inexistente o ruta con 404)
Verde	Se escribe el código mínimo	Toda la suite pasa
Refactor	Se limpia sin cambiar el comportamiento ni tocar tests	La suite sigue en verde

Agente	Función	Permisos
tdd	Guía el ciclo y hace un commit por fase (test(rojo), feat(verde), refactor)	Lee y escribe
revisor	Revisa seguridad y robustez del diff con cinco preguntas fijas y un formato de hallazgo (TP 6 y TP 7)	Solo lectura

Evidencia: dos ciclos completos (resumen y tareas próximas) con las salidas reales de los tests: 14, 18 y 31 tests en verde al cierre de cada etapa. Un commit por fase en el historial de git. Si el revisor encuentra algo, el hallazgo vuelve al inicio del ciclo: primero un test que lo muestre.

Documentos: CLAUDE.md (contexto para agentes), docs/FLUJO-DE-TRABAJO.md, docs/TDD-ROJO-VERDE-REFACTOR.md y docs/revisiones/.

13. Estructura del repositorio

```
TareasYA/
|-- README.md, CLAUDE.md
|-- .claude/agents/ revisor.md, tdd.md
|-- backend/
|   |-- src/ app.js, server.js, routes/, controllers/,
|   |        services/, repositories/, logic/
|   |-- test/ api, resumen, proximas (+ helpers)
|   |-- openapi.yaml, requests.http
|   `-- data/ (tareas.json se crea solo; ignorado por git)
|-- frontend/
|   |-- index.html, vite.config.js
|   `-- src/ App.jsx, main.jsx, styles.css,
|            components/, logic/ (+ tests)
`-- docs/ SPEC, BITACORA, API.md, FLUJO-DE-TRABAJO.md,
          TDD-ROJO-VERDE-REFACTOR.md, revisiones/, entregas/
```

14. Ejecución

```
# Backend (puerto 3001)
cd backend && npm install && npm start
npm test

# Frontend (puerto 5173, con proxy a /api)
cd frontend && npm install && npm run dev
npm test
npm run build      # genera frontend/dist
```

Si el backend no está en ejecución, el frontend arranca igual y funciona en modo local.

15. Deuda técnica y riesgos

#	Tema	Impacto	Acción propuesta
1	El JSON completo se lee y reescribe en cada petición	Rinde mal con miles de tareas	Migrar a SQLite con índices y paginación
2	La interfaz filtra y resume en el navegador aunque la API ya filtra y tiene /resumen y /proximas	Trabajo duplicado con listas grandes	Pasar filtros por query string a la API
3	Sin autenticación	Cualquiera con acceso ve todo	Cuentas y sesiones antes de publicar
4	Sin tests de componentes ni de punta a punta	Errores de UI no detectados automáticamente	Testing Library y Playwright
5	La interfaz no se probó manualmente en un navegador	Posibles detalles visuales sin pulir	Prueba manual en escritorio y celular
6	Datos del modo local no se sincronizan con el servidor	Tareas locales no aparecen al volver el servidor	Importar datos locales al detectar la API

16. Hoja de ruta

Versión	Contenido	Criterio de aceptación
v0.12	El frontend usa /resumen y /proximas; tests de componentes y prueba manual en navegador y celular	Flujos de alta, edición y borrado verificados
v0.13	SQLite, índices, paginación, filtros en servidor	Respuesta estable con 10.000 tareas
v0.14	Autenticación y tareas por usuario	Un usuario no puede ver tareas de otro
v1.0	Despliegue, PWA y recordatorios configurables	Instalable y usable sin conexión